



TCC70xx MCU BSP

API Specification

for Controller Area Network Controller

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	4
2	Terms and Abbreviations	4
3	Source Files.....	5
3.1	Location of Source Files.....	5
3.2	Simple Description of Source Files.....	5
4	Enumerations	7
4.1	CANFlagValue_t.....	7
4.2	CANMode_t.....	8
4.3	CANIdType_t.....	9
4.4	CANTxBufferMode_t.....	10
4.5	CANTxInterruptType_t	11
4.6	CANMessageBufferType_t.....	12
4.7	CANBufferDataField_t	13
4.8	CANRxOperationMode_t.....	14
4.9	CANFilterType_t.....	15
4.10	CANFilterConfig_t	16
4.11	CANDataLength_t.....	17
4.12	CANBitRateTiming_t.....	19
4.13	CANRingBufferType_t.....	20
4.14	CANErrorType_t.....	21
5	Structures	23
5.1	CANTimingParam_t.....	23
5.2	CANTxBuffer_t	24
5.3	CANRxBuffer_t	25
5.4	CANIdFilterList_t.....	26
5.5	CANIdFilter_t	27
5.6	CANMessage_t	28
5.7	CANTxEvent_t	29
5.8	CANCallbackFunc_t.....	30
6	Callback Functions	31
6.1	CANNotifyTxEventCB.....	31
6.2	CANNotifyRxEventCB	32
6.3	CANNotifyErrorEventCB	33
7	APIs	34
7.1	CAN_Init.....	34
7.2	CAN_Deinit	35
7.3	CAN_InitMessage	36
7.4	CAN_CheckNewRxMessage.....	37
7.5	CAN_GetNewRxMessage	38
7.6	CAN_SendMessage	39
7.7	CAN_RequestTxMessageCancellation	40
7.8	CAN_CheckNewTxEvent	41
7.9	CAN_GetNewTxEvent.....	42
7.10	CAN_SetControllerMode	43
7.11	CAN_GetControllerMode.....	44
7.12	CAN_GetProtocolStatus	45
7.13	CAN_GetControllerInfo	46
7.14	CAN_RegisterCallbackFunctionTx	47
7.15	CAN_RegisterCallbackFunctionRx	48
7.16	CAN_RegisterCallbackFunctionError.....	49
8	How to use CAN Driver	50
8.1	Initialize CAN Driver.....	50
8.1.1	Clock and Bit Timings	51
8.1.2	GPIO.....	51
8.1.3	Message RAM	52
8.1.4	Message ID Filters.....	53
8.2	Send and Receive Messages	54
9	References	56
10	Revision History	57
	Rev. 0.10: 2021-09-03	57

Figures

Figure 3.1 Location of Source Files 5

Figure 8.1 CAN Hardware Layer50

Figure 8.2 Message RAM Layout.....52

Figure 8.3 Rx Operation Flow.....55

Tables

Table 2.1 Terms and Abbreviation 4

1 INTRODUCTION

This document describes the controller area network controller device driver (hereinafter referred to as CAN driver) of TCC70xx MCU BSP. For more detailed information of the CAN controller in the TCC70xx MCU Sub-system, refer to Chapter 8 Control Area Network (CAN) Controller in "*TCC70xx Full Specification*". [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

CAN	Controller Area Network
CAN FD	CAN Flexible Data-rate
CAN controller	CAN IP of TCC70xx
CAN transceiver	External IC to control CAN signal (CAN Low/CAN High)
CRC	Cyclic Redundancy Check
H/W	Hardware
S/W	Software
TX	Transmitter
RX	Receiver
OSI 7 layer	Open System Interconnection 7 Layer

3 SOURCE FILES

3.1 Location of Source Files

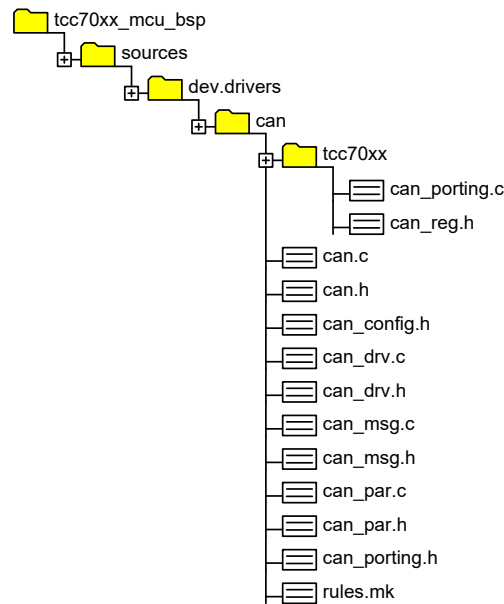


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `can_porting.c`
 - CAN driver porting source to be applied as user environment
- `can_reg.h`
 - Definition header of CAN controller registers
- `can.c`
 - CAN driver interface source for communication between application and driver
- `can.h`
 - CAN driver interface header for communication between application and driver
- `can_config.h`
 - CAN driver configuration header
- `can_drv.c`
 - CAN driver source to control CAN controller
- `can_drv.h`
 - CAN driver header to control CAN controller
- `can_msg.c`
 - CAN driver source to handle CAN messages
- `can_msg.h`
 - CAN driver header to handle CAN messages

- can_par.c
 - CAN driver setting parameter source
- can_par.h
 - CAN driver setting parameter header
- can_porting.h
 - CAN driver porting header to be applied as user environment
- rules.mk
 - Rules used for build

4 ENUMERATIONS

4.1 CANFlagValue_t

Enumeration variables of CAN flag value to check notification from CAN operation

Declaration

```
typedef enum CANFlagValue
{
    CAN_FLAG_FALSE    = 0,
    CAN_FLAG_TRUE     = 1,
    CAN_FLAG_MAX      = 2
} CANFlagValue_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>CAN_FLAG_FALSE</i>	<i>CAN flag value FALSE</i>
<i>CAN_FLAG_TRUE</i>	<i>CAN flag value TRUE</i>
<i>CAN_FLAG_MAX</i>	<i>Maximum flag value</i>

Remark

None

4.2 CANMode_t

Enumeration variables for current CAN controller modes

Declaration

```
typedef enum CANMode
{
    CAN_MODE_NO_INITIALIZATION    = 0,
    CAN_MODE_OPERATION            = 1,
    CAN_MODE_SLEEP                = 2,
    CAN_MODE_WAKE_UP              = 3,
    CAN_MODE_MONITORING           = 4,
    CAN_MODE_INTERNAL_TEST        = 5,
    CAN_MODE_EXTERNAL_TEST        = 6,
    CAN_MODE_RESET_CONTROLLER     = 7,
    CAN_MODE_MAX                  = 8
} CANMode_t;
```

Description

Value	Description
CAN_MODE_NO_INITIALIZATION	CAN controller is <u>not</u> initialized.
CAN_MODE_OPERATION	CAN controller is working.
CAN_MODE_SLEEP	CAN controller is in the sleep mode (requested clock stop).
CAN_MODE_WAKE_UP	CAN controller is in the wakeup mode (no requested clock stop).
CAN_MODE_MONITORING	CAN controller is in the bus monitoring mode.
CAN_MODE_INTERNAL_TEST	CAN controller is in the internal loop back test mode.
CAN_MODE_EXTERNAL_TEST	CAN controller is in the external loop back test mode.
CAN_MODE_RESET_CONTROLLER	CAN controller is reset (re-initialized).
CAN_MODE_MAX	Maximum CAN controller mode

Remark

None

4.3 CANIdType_t

Enumeration variables for CAN message ID types

Declaration

```
typedef enum CANIdType
{
    CAN_ID_TYPE_STANDARD = 0,
    CAN_ID_TYPE_EXTENDED = 1,
    CAN_ID_TYPE_MAX       = 2
} CANIdType_t;
```

Description

Value	Description
CAN_ID_TYPE_STANDARD	Standard message ID type
CAN_ID_TYPE_EXTENDED	Extended message ID type
CAN_ID_TYPE_MAX	Maximum message ID type

Remark

None

4.4 CANTxBufferMode_t

Enumeration variables for CAN Tx buffer modes

Declaration

```
typedef enum CANTxBufferMode
{
    CAN_TX_BUFFER_MODE_FIFO    = 0,
    CAN_TX_BUFFER_MODE_QUEUE   = 1
} CANTxBufferMode_t;
```

Description

Value	Description
CAN_TX_BUFFER_MODE_FIFO	Each message is sent via Tx FIFO one at a time.
CAN_TX_BUFFER_MODE_QUEUE	Messages are sent via Tx Queue at once.

Remark

None

4.5 CANTxInterruptType_t

Enumeration variables for CAN Tx interrupt types

Declaration

```
typedef enum CANTxInterruptType
{
    CAN_TX_INT_TYPE_TRANSMIT_COMPLETED           = 0,
    CAN_TX_INT_TYPE_TRANSMIT_CANCEL_FINISHED     = 1,
    CAN_TX_INT_TYPE_NEW_EVENT                    = 2
} CANTxInterruptType_t;
```

Description

Value	Description
CAN_TX_INT_TYPE_TRANSMIT_COMPLETED	Interrupt is generated when a message transmission is completed.
CAN_TX_INT_TYPE_TRANSMIT_CANCEL_FINISHED	Interrupt is generated when canceling a message transmission is completed.
CAN_TX_INT_TYPE_NEW_EVENT	Interrupt is generated when a Tx event message arrives.

Remark

None

4.6 CANMessageBufferType_t

Enumeration variables for CAN buffer types

Declaration

```
typedef enum CANMessageBufferType
{
    CAN_RX_BUFFER_TYPE_DBUFFER    = 0,
    CAN_RX_BUFFER_TYPE_FIFO0      = 1,
    CAN_RX_BUFFER_TYPE_FIFO1      = 2,
    CAN_TX_BUFFER_TYPE_FIFO        = 3,
    CAN_TX_BUFFER_TYPE_QUEUE       = 4,
    CAN_TX_BUFFER_TYPE_DBUFFER     = 5
} CANMessageBufferType_t;
```

Description

Value	Description
CAN_RX_BUFFER_TYPE_DBUFFER	Rx message is stored in dedicated buffer.
CAN_RX_BUFFER_TYPE_FIFO0	Rx message is stored in Rx FIFO 0.
CAN_RX_BUFFER_TYPE_FIFO1	Rx message is stored in Rx FIFO 1.
CAN_TX_BUFFER_TYPE_FIFO	Tx message is sent via Tx FIFO.
CAN_TX_BUFFER_TYPE_QUEUE	Tx message is sent via Tx Queue.
CAN_TX_BUFFER_TYPE_DBUFFER	Tx message is sent via Tx buffer.

Remark

None

4.7 CANBufferDataField_t

Enumeration variables for CAN buffer data size

Declaration

```
typedef enum CANBufferDataField
{
    CAN_BUFFER_DATA_FIELD_8      = 0,
    CAN_BUFFER_DATA_FIELD_12     = 1,
    CAN_BUFFER_DATA_FIELD_16     = 2,
    CAN_BUFFER_DATA_FIELD_20     = 3,
    CAN_BUFFER_DATA_FIELD_24     = 4,
    CAN_BUFFER_DATA_FIELD_32     = 5,
    CAN_BUFFER_DATA_FIELD_48     = 6,
    CAN_BUFFER_DATA_FIELD_64     = 7,
    CAN_BUFFER_DATA_FIELD_MAX    = 8
} CANBufferDataField_t;
```

Description

Value	Description
CAN_BUFFER_DATA_FIELD_8	Message data field size is 8 bytes.
CAN_BUFFER_DATA_FIELD_12	Message data field size is 12 bytes.
CAN_BUFFER_DATA_FIELD_16	Message data field size is 16 bytes.
CAN_BUFFER_DATA_FIELD_20	Message data field size is 20 bytes.
CAN_BUFFER_DATA_FIELD_24	Message data field size is 24 bytes.
CAN_BUFFER_DATA_FIELD_32	Message data field size is 32 bytes.
CAN_BUFFER_DATA_FIELD_48	Message data field size is 48 bytes.
CAN_BUFFER_DATA_FIELD_64	Message data field size is 64 bytes.
CAN_BUFFER_DATA_FIELD_MAX	Maximum message data field size

Remark

None

4.8 CANRxOperationMode_t

Enumeration variables for CAN Rx operation modes

Declaration

```
typedef enum CANRxOperationMode
{
    CAN_RX_OP_BLOCKING    = 0,
    CAN_RX_OP_OVERWRITE   = 1,
    CAN_RX_OP_MAX         = 2
} CANRxOperationMode_t;
```

Description

Value	Description
CAN_RX_OP_BLOCKING	If Rx FIFO is full, no further messages are written.
CAN_RX_OP_OVERWRITE	If Rx FIFO is full, the next message is overwritten on the oldest message.
CAN_RX_OP_MAX	Maximum CAN Rx operation mode

Remark

None

4.9 CANFilterType_t

Enumeration variables for CAN Rx message filter types

Declaration

```
typedef enum CANFilterType
{
    CAN_FILTER_TYPE_RANGE           = 0,
    CAN_FILTER_TYPE_DUAL            = 1,
    CAN_FILTER_TYPE_CLASSIC         = 2,
    CAN_FILTER_TYPE_ELEMENTDISABLED = 3,
    CAN_FILTER_TYPE_MAX             = 4
} CANFilterType_t;
```

Description

Value	Description
CAN_FILTER_TYPE_RANGE	Rx message is accepted if ID is in the designated range.
CAN_FILTER_TYPE_DUAL	Rx message is accepted if ID is matched to designated ID.
CAN_FILTER_TYPE_CLASSIC	Rx message is accepted if ID is matched to designated patterns.
CAN_FILTER_TYPE_ELEMENTDISABLED	Rx message filter element is disabled.
CAN_FILTER_TYPE_MAX	Maximum Rx message filter types

Remark

None

4.10 CANFilterConfig_t

Enumeration variables for CAN filter configurations. These enumeration variables indicate where Rx message is stored if it is accepted.

Declaration

```
typedef enum CANFilterConfig
{
    CAN_FILTER_CFG_DISABLE_ELEMENT = 0,
    CAN_FILTER_CFG_RXFIFO0         = 1,
    CAN_FILTER_CFG_RXFIFO1         = 2,
    CAN_FILTER_CFG_REJECT_ID       = 3,
    CAN_FILTER_CFG_PRY             = 4,
    CAN_FILTER_CFG_PRY_N_RXFIFO0   = 5,
    CAN_FILTER_CFG_PRY_N_RXFIFO1   = 6,
    CAN_FILTER_CFG_RXBUFFER        = 7,
    CAN_FILTER_CONFIG_MAX          = 8
} CANFilterConfig_t;
```

Description

Value	Description
<code>CAN_FILTER_CFG_DISABLE_ELEMENT</code>	Message filter element is disabled.
<code>CAN_FILTER_CFG_RXFIFO0</code>	Message is stored in Rx FIFO 0 if it is accepted.
<code>CAN_FILTER_CFG_RXFIFO1</code>	Message is stored in Rx FIFO 1 if it is accepted.
<code>CAN_FILTER_CFG_REJECT_ID</code>	Message is rejected (not stored) if it is accepted.
<code>CAN_FILTER_CFG_PRY</code>	Message is set to priority if it is accepted.
<code>CAN_FILTER_CFG_PRY_N_RXFIFO0</code>	Message is set to priority and stored in Rx FIFO 0 if it is accepted.
<code>CAN_FILTER_CFG_PRY_N_RXFIFO1</code>	Message is set to priority and stored in Rx FIFO 1 if it is accepted.
<code>CAN_FILTER_CFG_RXBUFFER</code>	Message is stored in dedicated Rx buffer if it is accepted.
<code>CAN_FILTER_CONFIG_MAX</code>	Maximum message filter configuration

Remark

None

4.11 CANDataLength_t

Enumeration variables for CAN message data size

Declaration

```
typedef enum CANDataLength
{
    CAN_DATA_LENGTH_0      = 0,
    CAN_DATA_LENGTH_1      = 1,
    CAN_DATA_LENGTH_2      = 2,
    CAN_DATA_LENGTH_3      = 3,
    CAN_DATA_LENGTH_4      = 4,
    CAN_DATA_LENGTH_5      = 5,
    CAN_DATA_LENGTH_6      = 6,
    CAN_DATA_LENGTH_7      = 7,
    CAN_DATA_LENGTH_8      = 8,
    CAN_DATA_LENGTH_12     = 9,
    CAN_DATA_LENGTH_16     = 10,
    CAN_DATA_LENGTH_20     = 11,
    CAN_DATA_LENGTH_24     = 12,
    CAN_DATA_LENGTH_32     = 13,
    CAN_DATA_LENGTH_48     = 14,
    CAN_DATA_LENGTH_64     = 15,
    CAN_DATA_LENGTH_MAX    = 16
} CANDataLength_t;
```

Description

Value	Description
CAN_DATA_LENGTH_0	Message data size is 0.
CAN_DATA_LENGTH_1	Message data size is 1.
CAN_DATA_LENGTH_2	Message data size is 2.
CAN_DATA_LENGTH_3	Message data size is 3.
CAN_DATA_LENGTH_4	Message data size is 4.
CAN_DATA_LENGTH_5	Message data size is 5.
CAN_DATA_LENGTH_6	Message data size is 6.
CAN_DATA_LENGTH_7	Message data size is 7.
CAN_DATA_LENGTH_8	Message data size is 8.
CAN_DATA_LENGTH_12	Message data size is 12.
CAN_DATA_LENGTH_16	Message data size is 16.
CAN_DATA_LENGTH_20	Message data size is 20.
CAN_DATA_LENGTH_24	Message data size is 24.

<i>Value</i>	<i>Description</i>
<i>CAN_DATA_LENGTH_32</i>	<i>Message data size is 32.</i>
<i>CAN_DATA_LENGTH_48</i>	<i>Message data size is 48.</i>
<i>CAN_DATA_LENGTH_64</i>	<i>Message data size is 64.</i>
<i>CAN_DATA_LENGTH_MAX</i>	<i>Maximum message data size</i>

Remark

None

4.12 CANBitRateTiming_t

Enumeration variables for CAN bit rates

Declaration

```
typedef enum CANBitRateTiming
{
    CAN_BIT_RATE_TIMING_ARBITRATION = 0,
    CAN_BIT_RATE_TIMING_DATA        = 1,
    CAN_BIT_RATE_TIMING_MAX         = 2
} CANBitRateTiming_t;
```

Description

Value	Description
CAN_BIT_RATE_TIMING_ARBITRATION	Nominal bit timing
CAN_BIT_RATE_TIMING_DATA	Data bit timing
CAN_BIT_RATE_TIMING_MAX	Maximum bit timing

Remark

None

4.13 CANRingBufferType_t

Enumeration variables for CAN Rx ring buffer types

Declaration

```
typedef enum CANRingBufferType
{
    CAN_RING_BUFFER_FIRST = 0,
    CAN_RING_BUFFER_LAST  = 1,
    CAN_RING_BUFFER_MAX   = 2
} CANRingBufferType_t;
```

Description

Value	Description
CAN_RING_BUFFER_FIRST	Rx priority message is copied to the first index of ring buffer.
CAN_RING_BUFFER_LAST	Rx message is copied to the last index of ring buffer.
CAN_RING_BUFFER_MAX	Maximum ring buffer type

Remark

None

4.14 CANErrorType_t

Enumeration variables for CAN error types

Declaration

```
typedef enum CANErrorType
{
    CAN_ERROR_NONE                = 0,
    CAN_ERROR_NOT_INIT            = 1,
    CAN_ERROR_BAD_PARAM           = 2,
    CAN_ERROR_ALLOC               = 3,
    CAN_ERROR_NO_MESSAGE         = 4,
    CAN_ERROR_FIFO_QUEUE_FULL     = 5,
    CAN_ERROR_FIFO_QUEUE_LOST     = 6,
    CAN_ERROR_CONTROLLER_MODE     = 7,
    CAN_ERROR_TIMEOUT            = 8,
    CAN_ERROR_NO_BUFFER           = 9,
    CAN_ERROR_INT_ACCESS_RESERVED_ADDRESS = 20,
    CAN_ERROR_INT_PROTOCOL        = 21,
    CAN_ERROR_INT_BUS_OFF         = 22,
    CAN_ERROR_INT_WARNING         = 23,
    CAN_ERROR_INT_PASSIVE         = 24,
    CAN_ERROR_INT_BIT             = 25,
    CAN_ERROR_INT_TIMEOUT         = 26,
    CAN_ERROR_INT_RAM_ACCESS_FAIL = 27,
    CAN_ERROR_INT_TX_EVENT_FULL   = 28,
    CAN_ERROR_INT_TX_EVENT_LOST   = 29,
    CAN_ERROR_CODE_STUFF          = 41,
    CAN_ERROR_CODE_FORM           = 42,
    CAN_ERROR_CODE_ACK            = 43,
    CAN_ERROR_CODE_BIT1           = 44,
    CAN_ERROR_CODE_BIT0           = 45,
    CAN_ERROR_CODE_CRC            = 46,
    CAN_ERROR_CODE_NO_CHANGE      = 47
} CANErrorType_t;
```

Description

Value	Description
CAN_ERROR_NONE	No errors
CAN_ERROR_NOT_INIT	CAN driver is <u>not</u> initialized.
CAN_ERROR_BAD_PARAM	Bad parameter is passed to APIs.
CAN_ERROR_ALLOC	CAN message RAM memory is <u>not</u> allocated.
CAN_ERROR_NO_MESSAGE	No message
CAN_ERROR_FIFO_QUEUE_FULL	CAN Rx FIFO 0/1 is full.
CAN_ERROR_FIFO_QUEUE_LOST	CAN Rx FIFO 0/1 message is lost.
CAN_ERROR_CONTROLLER_MODE	Unexpected CAN controller mode

Value	Description
<i>CAN_ERROR_TIMEOUT</i>	<i>Timeout error in CAN driver</i>
<i>CAN_ERROR_NO_BUFFER</i>	<i>Ring buffer is invalid.</i>
<i>CAN_ERROR_INT_ACCESS_RESERVED_ADDRESS</i>	<i>Access reserved address of CAN controller registers.</i>
<i>CAN_ERROR_INT_PROTOCOL</i>	<i>CAN Protocol error occurs in CAN controller.</i>
<i>CAN_ERROR_INT_BUS_OFF</i>	<i>CAN controller is in the bus off status.</i>
<i>CAN_ERROR_INT_WARNING</i>	<i>CAN controller is in the bus warning status.</i>
<i>CAN_ERROR_INT_PASSIVE</i>	<i>Passive error occurs in CAN controller.</i>
<i>CAN_ERROR_INT_BIT</i>	<i>Bit error occurs in CAN controller.</i>
<i>CAN_ERROR_INT_TIMEOUT</i>	<i>Timeout error occurs in CAN controller. It indicates that CAN driver does <u>not</u> get an Rx message from FIFO0/1 or dedicated buffer until designated time.</i>
<i>CAN_ERROR_INT_RAM_ACCESS_FAIL</i>	<i>Message RAM access fail error occurs in CAN controller.</i>
<i>CAN_ERROR_INT_TX_EVENT_FULL</i>	<i>Tx event FIFO is full.</i>
<i>CAN_ERROR_INT_TX_EVENT_LOST</i>	<i>Tx event FIFO message is lost.</i>
<i>CAN_ERROR_CODE_STUFF</i>	<i>CAN stuff error in protocol occurs.</i>
<i>CAN_ERROR_CODE_FORM</i>	<i>CAN format error in protocol occurs.</i>
<i>CAN_ERROR_CODE_ACK</i>	<i>CAN ack error in protocol occurs.</i>
<i>CAN_ERROR_CODE_BIT1</i>	<i>CAN bit1 error in protocol occurs.</i>
<i>CAN_ERROR_CODE_BIT0</i>	<i>CAN bit0 error in protocol occurs.</i>
<i>CAN_ERROR_CODE_CRC</i>	<i>CAN CRC error in protocol occurs.</i>
<i>CAN_ERROR_CODE_NO_CHANGE</i>	<i>CAN no change in protocol occurs.</i>

Remark

None

5 STRUCTURES

5.1 CANTimingParam_t

Structure variables for CAN bit rates

Declaration

```
typedef struct CANTimingParam
{
    uint8    tpBRP;
    uint8    tpPROP;
    uint8    tpPhaseSeg1;
    uint8    tpPhaseSeg2;
    uint8    tpSJW;
    uint8    tpTDC;
    uint8    tpTDCOffset;
    uint8    tpTDCFilterWindow;
} CANTimingParam_t;
```

Description

Value	Description
<i>tpBRP</i>	<i>Pre-scaler value</i>
<i>tpPROP</i>	<i>Propagation segment value</i>
<i>tpPhaseSeg1</i>	<i>Time segment value before sample point</i>
<i>tpPhaseSeg2</i>	<i>Time segment value after sample point</i>
<i>tpSJW</i>	<i>Data Synchronization Jump Width value</i>
<i>tpTDC</i>	<i>Enable or disable transmitter delay compensation</i>
<i>tpTDCOffset</i>	<i>Transmitter delay compensation offset value</i>
<i>tpTDCFilterWindow</i>	<i>Transmitter delay compensation filter window length</i>

Remark

None

5.2 CANTxBuffer_t

Structure variables for CAN Tx Buffer

Declaration

```
typedef struct CANTxBuffer
{
    uint8          tbNumberOfDedicatedBuffer;
    uint8          tbNumberOfFIFOorQueue;
    CANTxBufferMode_t  tbTxFIFOorQueueMode;
    CANBufferDataField_t  tbBufferElementSize;
    uint32         tbInterruptConfig;
} CANTxBuffer_t;
```

Description

Value	Description
tbNumberOfDedicatedBuffer	The number of Tx buffers
tbNumberOfFIFOorQueue	The number of Tx FIFO or Queue
tbTxFIFOorQueueMode	Selected Tx buffer mode is FIFO or Queue. Refer to CANTxBufferMode t .
tbBufferElementSize	Size of Tx buffer element. Refer to CANBufferDataField t .
tbInterruptConfig	Configuration of Tx interrupt

Remark

None

5.3 CANRxBuffer_t

Structure variables for CAN Rx Buffer

Declaration

```
typedef struct CANRxBuffer
{
    uint8            rbNumberOfElement;
    CANBufferDataField_t  rbDataFieldSize;
    CANRxOperationMode_t  rbOperationMode;
} CANRxBuffer_t;
```

Description

Value	Description
<i>rbNumberOfElement</i>	<i>The number of Rx buffers</i>
<i>rbDataFieldSize</i>	<i>Size of Rx buffer element. Refer to CANBufferDataField t.</i>
<i>rbOperationMode</i>	<i>Operation mode of Rx buffer. Refer to CANRxOperationMode t.</i>

Remark

None

5.4 CANIdFilterList_t

Structure variables for CAN ID filter list

Declaration

```
typedef struct CANIdFilterList
{
    CANFilterType_t      idflFilterType;
    CANFilterConfig_t    idflFilterElementConfiguration;
    uint32               idflFilterID1;
    uint32               idflFilterID2;
} CANIdFilterList_t;
```

Description

Value	Description
idflFilterType	ID filter type. Refer to CANFilterType t.
idflFilterElementConfiguration	Configuration of ID filter element. Refer to CANFilterConfig t.
idflFilterID1	Filter ID parameter 1
idflFilterID2	Filter ID parameter 2

Remark

None

5.5 CANIdFilter_t

Structure variables for CAN ID filter

Declaration

```
typedef struct CANIdFilter
{
    CANIdFilterList_t *   idfList[CAN_CONTROLLER_NUMBER];
} CANIdFilter_t;
```

Description

Value	Description
idfList	CAN ID filter list. Refer to CANIdFilterList t .

Remark

None

5.6 CANMessage_t

Structure variables for CAN messages

Declaration

```
typedef struct CANMessage
{
    CANMessageBufferType_t    mBufferType;
    uint8                     mBufferIndex;
    uint8                     mErrorStateIndicator;
    uint8                     mExtendedId;
    uint8                     mRemoteTransmitRequest;
    uint32                    mId;
    uint8                     mFDFormat;
    uint8                     mBitRateSwitching;
    uint8                     mMessageMarker;
    uint8                     mEventFIFOControl;
    uint8                     mDataLength;
    uint8                     mData[CAN_DATA_LENGTH_SIZE];
} CANMessage_t;
```

Description

Value	Description
<i>mBufferType</i>	Buffer types. Refer to CANMessageBufferType_t .
<i>mBufferIndex</i>	Current buffer index
<i>mErrorStateIndicator</i>	Error state indicator in CAN controller
<i>mExtendedId</i>	Value that indicates standard ID or extended ID
<i>mRemoteTransmitRequest</i>	Value that indicates whether remote transmit request or not
<i>mId</i>	Message ID
<i>mFDFormat</i>	Value that indicates whether the message is FD format or not
<i>mBitRateSwitching</i>	Value that indicates whether bit rate switch is included in the message or not
<i>mMessageMarker</i>	Message marker value
<i>mEventFIFOControl</i>	Value that indicates whether Tx event FIFO is used or not
<i>mDataLength</i>	Message data Length
<i>mData</i>	Message data

Remark

None

5.7 CANTxEvt_t

Structure variables for CAN Tx event messages

Declaration

```
typedef struct CANTxEvt
{
    uint8      teMessageMarker;
    uint8      teFDFormat;
    uint8      teBitRateSwitching;
    uint8      teDataLength;
    uint8      teExtendedId;
    uint8      teRemoteTransmitRequest;
    uint32     teId;
} CANTxEvt_t;
```

Description

Value	Description
<i>teMessageMarker</i>	<i>Tx event message marker value</i>
<i>teFDFormat</i>	<i>Value that indicates whether the message is FD format or not</i>
<i>teBitRateSwitching</i>	<i>Value that indicates whether bit rate switch is included in the message or not</i>
<i>teDataLength</i>	<i>Message data Length</i>
<i>teExtendedId</i>	<i>Value that indicates standard ID or extended ID</i>
<i>teRemoteTransmitRequest</i>	<i>Value that indicates whether remote transmit request or not</i>
<i>teId</i>	<i>Message ID</i>

Remark

None

5.8 CANCallbackFunc_t

Structure variables for CAN callback functions

Declaration

```
typedef struct CANCallbackFunc
{
    CANNotifyTxEventCB      cbNotifyTxEvent;
    CANNotifyRxEventCB      cbNotifyRxEvent;
    CANNotifyErrorEventCB   cbNotifyErrorEvent;
} CANCallbackFunc_t;
```

Description

Value	Description
cbNotifyTxEvent	Callback function for Tx. Refer to CANNotifyTxEventCB .
cbNotifyRxEvent	Callback function for Rx. Refer to CANNotifyRxEventCB .
cbNotifyErrorEvent	Callback function for errors. Refer to CANNotifyErrorEventCB .

Remark

None

6 CALLBACK FUNCTIONS

6.1 CANNotifyTxEventCB

Callback function for notifying Tx

Declaration

```
typedef void (*CANNotifyTxEventCB)
(
    uint8          ucCh,
    CANTxInterruptType_t  uiIntType
);
```

Parameters

Value	Description
ucCh	Channel number value for each CAN controller channel
uiIntType	Tx interrupt types. Refer to CANTxInterruptType t .

Return

None

Remark

None

6.2 CANNotifyRxEventCB

Callback function for notifying Rx

Declaration

```
typedef void (*CANNotifyRxEventCB)
(
    uint8          ucCh,
    uint32         uiRxIndex,
    uint32         uiRxBufferType,
    CANErrorType_t uiError
);
```

Parameters

Value	Description
ucCh	Channel number value for each CAN controller channel
uiRxIndex	Index of Rx buffer
uiRxBufferType	Type of Rx buffer
uiError	CAN controller errors. Refer to CANErrorType t .

Return

None

Remark

None

6.3 CANNotifyErrorEventCB

Callback functions for notifying error in CAN controller

Declaration

```
typedef void (*CANNotifyErrorEventCB)
(
    uint8      ucCh,
    CANErrorType_t  uiError
);
```

Parameters

Value	Description
ucCh	Channel number value for each CAN controller channel
uiError	CAN controller errors. Refer to CANErrorType_t .

Return

None

Remark

None

7 APIs

7.1 CAN_Init

Initializes the CAN controller, memory, and GPIO of each channel.

Declaration

```
CANErrorType_t CAN_Init
(
    void
);
```

Parameters

None

Return

Value	Description
CAN_ERROR_NONE	Success. Refer to CANErrorType t .
Others	Failure. Refer to CANErrorType t .

Remark

None

7.2 CAN_Deinit

De-initializes the CAN controller and memory of all channels.

Declaration

```
CANErrorType_t CAN_Deinit
(
    void
);
```

Parameters

None

Return

Value	Description
CAN_ERROR_NONE	Success. Refer to CANErrorType t .
Others	Failure. Refer to CANErrorType t .

Remark

None

7.3 CAN_InitMessage

Initializes the CAN message buffer of each channel.

Declaration

```
CANErrorType_t CAN_InitMessage  
(  
    uint8      ucCh  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in]</i> CAN controller channel

Return

Value	Description
<i>CAN_ERROR_NONE</i>	Success. Refer to CANErrorType_t .
<i>Others</i>	Failure. Refer to CANErrorType_t .

Remark

None

7.4 CAN_CheckNewRxMessage

Checks whether there are newly received messages or not.

Declaration

```
uint32 CAN_CheckNewRxMessage  
(  
    uint8    ucCh  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] CAN controller channel</i>

Return

Value	Description
<i>Greater than 0</i>	<i>The number of new Rx messages</i>
<i>0</i>	<i>No message</i>

Remark

None

7.5 CAN_GetNewRxMessage

Gets the newly received message.

Declaration

```
CANErrorType_t CAN_GetNewRxMessage  
(  
    uint8          ucCh,  
    CANMessage_t * psMsg  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in]</i> CAN controller channel
<i>psMsg</i>	<i>[out]</i> Newly received message data. Refer to CANMessage_t .

Return

Value	Description
<i>CAN_ERROR_NONE</i>	Success. Refer to CANErrorType_t .
<i>Others</i>	Failure. Refer to CANErrorType_t .

Remark

None

7.6 CAN_SendMessage

Sends a message.

Declaration

```
CANErrorType_t CAN_SendMessage  
(  
    uint8          ucCh,  
    const CANMessage_t * psMsg  
    uint8 *        pucTxBufferIndex,  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in]</i> CAN controller channel
<i>psMsg</i>	<i>[in]</i> Message data to be transmitted. Refer to CANMessage_t .
<i>pucTxBufferIndex</i>	<i>[out]</i> Index of Tx buffer

Return

Value	Description
<i>CAN_ERROR_NONE</i>	Success. Refer to CANErrorType_t .
<i>Others</i>	Failure. Refer to CANErrorType_t .

Remark

None

7.7 CAN_RequestTxMessageCancellation

Cancels a Tx message.

Declaration

```
CANErrorType_t CAN_RequestTxMessageCancellation  
(  
    uint8      ucCh,  
    uint8      ucTxIndex  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] CAN controller channel</i>
<i>ucTxIndex</i>	<i>[in] Index of currently being transmitted message</i>

Return

Value	Description
<i>CAN_ERROR_NONE</i>	Success. Refer to CANErrorType t .
<i>Others</i>	Failure. Refer to CANErrorType t .

Remark

None

7.8 CAN_CheckNewTxEvent

Checks whether there are newly transmitted event messages or not.

Declaration

```
uint32 CAN_CheckNewTxEvent
(
    uint8    ucCh
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] CAN controller channel</i>

Return

Value	Description
<i>Greater than 0</i>	<i>The number of newly transmitted event messages</i>
<i>0</i>	<i>No message</i>

Remark

None

7.9 CAN_GetNewTxEvent

Gets the newly transmitted event message.

Declaration

```
CANErrorType_t CAN_GetNewTxEvent
(
    uint8          ucCh,
    CANTxEvent_t * psTxEvent
);
```

Parameters

Value	Description
ucCh	[in] CAN controller channel
psTxEvent	[out] Newly transmitted event message data. Refer to CANTxEvent t.

Return

Value	Description
CAN_ERROR_NONE	Success. Refer to CANErrorType t.
Others	Failure. Refer to CANErrorType t.

Remark

None

7.10 CAN_SetControllerMode

Sets the CAN controller mode.

Declaration

```
CANErrorType_t CAN_SetControllerMode  
(  
    uint8      ucCh,  
    CANMode_t  ucControllerMode  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in]</i> CAN controller channel
<i>ucControllerMode</i>	<i>[in]</i> CAN controller mode. Refer to CANMode_t .

Return

Value	Description
<i>CAN_ERROR_NONE</i>	Success. Refer to CANErrorType_t .
<i>Others</i>	Failure. Refer to CANErrorType_t .

Remark

None

7.11 CAN_GetControllerMode

Gets the current CAN controller mode.

Declaration

```
CANMode_t CAN_GetControllerMode
(
    uint8      ucCh
);
```

Parameters

Value	Description
ucCh	[in] CAN controller channel

Return

Value	Description
CANMode_t	Refer to CANMode_t .

Remark

None

7.12 CAN_GetProtocolStatus

Gets the current protocol status of CAN controller.

Declaration

```
uint32 CAN_GetProtocolStatus
(
    uint8      ucCh
);
```

Parameters

Value	Description
ucCh	[in] CAN controller channel

Return

Value	Description
Any values	Current protocol status value

Remark

None

7.13 CAN_GetControllerInfo

Gets CAN controller information.

Declaration

```
void *CAN_GetControllerInfo  
(  
    uint8    ucCh  
);
```

Parameters

Value	Description
<i>ucCh</i>	<i>[in] CAN controller channel</i>

Return

Value	Description
<i>Not NULL</i>	<i>Pointer of CAN controller information of designated channel</i>
<i>NULL</i>	<i>Failure</i>

Remark

None

7.14 CAN_RegisterCallbackFunctionTx

Registers the callback function to notify that CAN Tx is completed.

Declaration

```
CANErrorType_t CAN_RegisterCallbackFunctionTx
(
    CANNotifyTxEventCB    pCbFnTx
);
```

Parameters

Value	Description
pCbFnTx	[in] Callback function address to notify that CAN Tx is completed. Refer to CANNotifyTxEventCB .

Return

Value	Description
CAN_ERROR_NONE	Success. Refer to CANErrorType t .
Others	Failure. Refer to CANErrorType t .

Remark

None

7.15 CAN_RegisterCallbackFunctionRx

Registers the callback function to notify CAN Rx.

Declaration

```
CANErrorType_t CAN_RegisterCallbackFunctionRx  
(  
    CANNotifyRxEventCB    pCbFnRx  
);
```

Parameters

Value	Description
<i>pCbFnRx</i>	[in] Callback function address to notify CAN Rx. Refer to CANNotifyRxEventCB .

Return

Value	Description
<i>CAN_ERROR_NONE</i>	Success. Refer to CANErrorType t .
<i>Others</i>	Failure. Refer to CANErrorType t .

Remark

None

7.16 CAN_RegisterCallbackFunctionError

Registers the callback function to notify the CAN error.

Declaration

```
CANErrorType_t CAN_RegisterCallbackFunctionError
(
    CANNotifyErrorEventCB    pCbFnError
);
```

Parameters

Value	Description
pCbFnError	[in] Callback function address to notify the CAN Error. Refer to CANNotifyErrorEventCB .

Return

Value	Description
CAN_ERROR_NONE	Success. Refer to CANErrorType t .
Others	Failure. Refer to CANErrorType t .

Remark

None

8 HOW TO USE CAN DRIVER

The CAN driver controls and sets the CAN controller. The CAN transceiver is not controlled by the CAN driver.

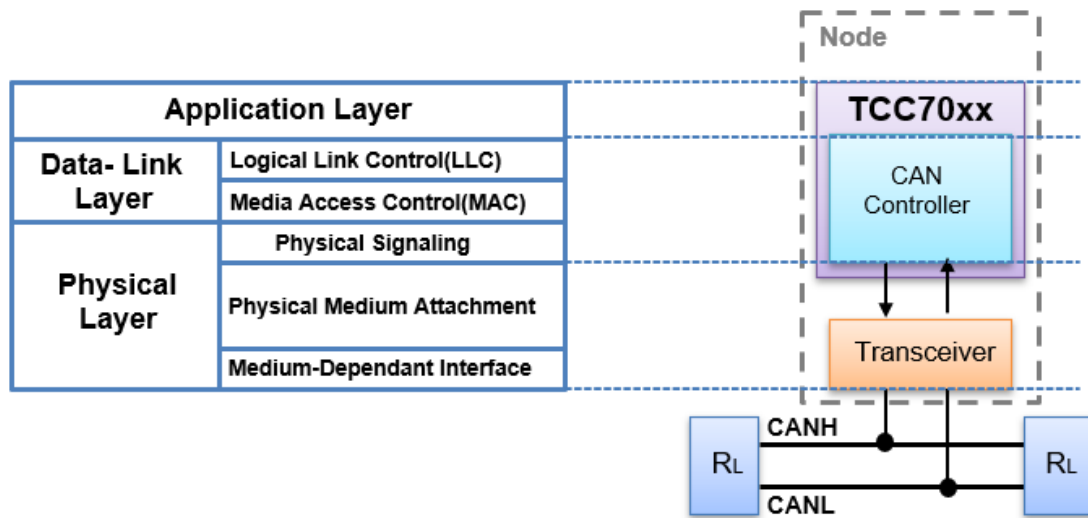


Figure 8.1 CAN Hardware Layer

The CAN controller and transceiver work as data-link and physical layer of OSI 7 layer.

8.1 Initialize CAN Driver

The CAN controller clock should be set properly for CAN driver operation. The CAN driver must set GPIO, message RAM, and message ID filters. These must be initialized by **CAN_Init()** function.

```
void main()
{
    CANErrorType_t ret;

    /* register callback functions */
    (void)CAN_RegisterCallbackFunctionTx(&CAN_DemoCallbackTxEvent);
    (void)CAN_RegisterCallbackFunctionRx(&CAN_DemoCallbackRxEvent);
    (void)CAN_RegisterCallbackFunctionError(&CAN_DemoCallbackErrorEvent);

    ret = CAN_Init();
    if(result == CAN_ERROR_NONE)
    {
        /* Success to initialize */
    }
    else
    {
        /* Fail to initialize */
    }
}
```

8.1.1 Clock and Bit Timings

The CAN driver basically sets the CAN controller clock to 40 MHz (Maximum 80 MHz).

```
vcp_mcu/sources/dev.drivers/can/can_config.h

#define CAN_CONTROLLER_CLOCK      (40000000) /* 40MHz */
```

The CAN driver sets the nominal bit timing to 500 kbits/s and the data bit timing to 2 Mbits/s.

Calculation for Nominal Bit Timing:

Nominal Bit Timing (NBTP) = (NTSEG1 + NTSEG2 + 3) * Tq

Tq = 25 ns (1/40 MHz) and NTSEG1 = 62, NTSEG2 = 15

NBTP = (62 + 15 + 3) * 25 ns = 2,000 ns, 1/2000 ns = 500 kbits/s

Calculation for Data Bit Timing:

Data Bit Timing (DBTP) = (DTSEG1 + DTSEG2 + 3) * Tq

Tq = 25 ns (1/40 MHz) and DTSEG1 = 12, DTSEG2 = 5

DBTP = (12 + 5 + 3) * 25 ns = 500 ns, 1/500 ns = 2 Mbits/s

Refer to the following bit timing parameters.

```
vcp_mcu/sources/dev.drivers/can/can_par.c

/* Arbitration Phase Timing Information */
CANTimingParam_t ArbitrationPhaseTimingPar[CAN_CONTROLLER_NUMBER] =
{
    /* BRP, PROP, TSEG1, TSEG2, SJW, TDC, TDCO, TDCF */
    {1, 47, 16, 16, 16, 0, 0, 0}, //channel 0, speed : 500Kbps / sample point : 80%
    {1, 47, 16, 16, 16, 0, 0, 0}, //channel 0, speed : 500Kbps / sample point : 80%
    {1, 47, 16, 16, 16, 0, 0, 0}, //channel 0, speed : 500Kbps / sample point : 80%
};

/* Data Phase Timing Information */
CANTimingParam_t DataPhaseTimingPar[CAN_CONTROLLER_NUMBER] =
{
    /* BRP, PROP, TSEG1, TSEG2, SJW, TDC, TDCO, TDCF */
    {1, 0, 13, 6, 6, 1, 14, 0}, //channel 0, speed : 2Mbps / sample point : 70%
    {1, 0, 13, 6, 6, 1, 14, 0}, //channel 0, speed : 2Mbps / sample point : 70%
    {1, 0, 13, 6, 6, 1, 14, 0}, //channel 0, speed : 2Mbps / sample point : 70%
};
```

8.1.2 GPIO

The CAN driver uses CAN Tx, Rx, and Standby (STB) port.

8.1.3 Message RAM

The CAN driver must configure message RAM to receive and send messages. Basically, the CAN driver uses internal SRAM for the message RAM.

Note: The message RAM should be configured in non-cacheable RAM area.

The following address shows the address of message RAM for CAN 0 channel.

If you use CAN 1 channel, its address starts from 0x00076000. If you use CAN 2 channel, its address starts from 0x00078000.

- Total: 0x1340 (4,928 bytes (1,232 words))
 - 0x00074000 ~ 0x00074040: Standard ID
 - 0x00074040 ~ 0x000740C0: Extended ID
 - 0x000740C0 ~ 0x00074540: Rx FIFO 0
 - 0x00074540 ~ 0x000749C0: Rx FIFO 1
 - 0x000749C0 ~ 0x00074E40: Rx dedicated buffer
 - 0x00074E40 ~ 0x000752C0: Tx Event FIFO
 - 0x000752C0 ~ 0x00075340: Tx Buffers

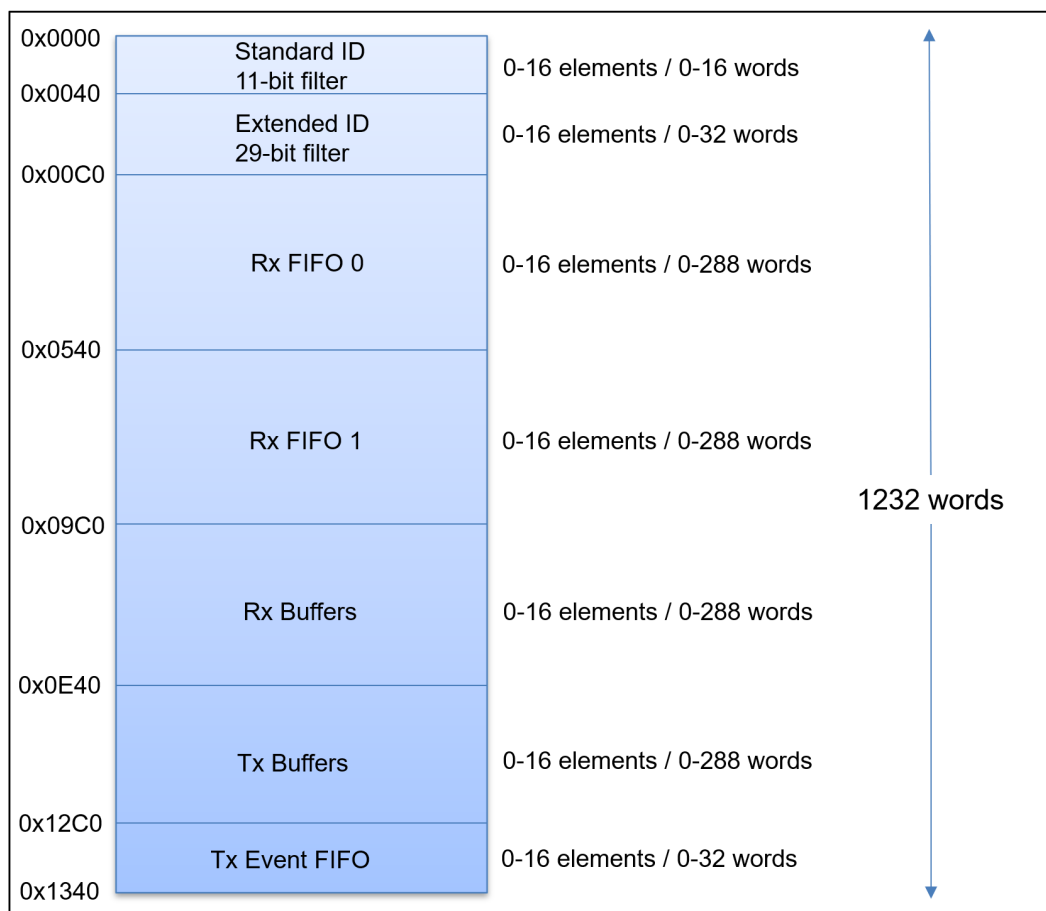


Figure 8.2 Message RAM Layout

8.1.4 Message ID Filters

The CAN driver decides to receive or reject a message via message filters. The messages are stored in the buffer that is configured in message filters. Basically, the message filter type is *range*. The range means that the message is received or rejected if message ID is between filter ID1 and filter ID2. Refer to the following message filter parameters.

vcp_mcu/sources/dev.drivers/can/can_par.c

```
static CANIdFilterList_t StandardIDFilterPar_Chx[CAN_STANDARD_ID_FILTER_NUMBER] =
{
    /* FilterType,          FilterElementConfiguration,    FilterID1,    FilterID2 */
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_RXFIFO0,        0x000,        0x100},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_RXFIFO1,        0x101,        0x200},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_PRY_N_RXFIFO0,   0x201,        0x300},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_PRY_N_RXFIFO1,   0x301,        0x400},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_RXBUFFER,        0x401,        0x0},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_REJECT_ID,       0x402,        0x7FF}
};

static CANIdFilterList_t ExtendedIDFilterPar_CH0[CAN_EXTENDED_ID_FILTER_NUMBER] =
{
    /* FilterType,          FilterElementConfiguration,    FilterID1,    FilterID2 */
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_RXFIFO0,        0x0000,       0x1000},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_RXFIFO1,        0x1001,       0x2000},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_PRY_N_RXFIFO0,   0x2001,       0x3000},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_PRY_N_RXFIFO1,   0x3001,       0x4000},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_RXBUFFER,        0x4001,       0x0},
    {CAN_FILTER_TYPE_RANGE, CAN_FILTER_CFG_REJECT_ID,       0x4002,       0x1FFFFFFF}
};
```

8.2 Send and Receive Messages

The CAN driver can send and receive messages by using CAN driver APIs. Refer to the following example code.

```
void main()
{
    CANErrorType_t ret;
    CANMessage_t *psTxMsg = NULL_PTR;
    CANMessage_t sRxMsg;
    CANTxEvent_t sTxEvtMsg;
    uint32 uiTxEvtMsgCnt;
    uint32 uiRxMsgCnt;
    uint8 ucTxBufferIndex;

    /* Initialized CAN driver */

    /* Send message */
    psTxMsg = &sTxMessageInfo[0];
    CAN_SendMessage(0, psTxMsg, &ucTxBufferIndex); //Channel 0 sends a predefined message

    /* Check Tx event message */
    uiTxEvtMsgCnt = CAN_CheckNewTxEvent(0);
    if(0UL < uiTxEvtMsgCnt)
    {
        /* Get & Check Tx event message */
        SAL_MemSet(&sTxEvtMsg, 0, sizeof(CANTxEvent_t));
        ret = CAN_GetNewTxEvent(0, &sTxEvtMsg); //Channel 0 gets a Tx event message
    }

    /* Receive message */
    uiRxMsgCnt = CAN_CheckNewRxMessage(1);
    if(0UL < uiRxMsgCnt)
    {
        Ret = CAN_GetNewRxMessage(1, &sRxMsg); //Channel 1 gets a Rx message
    }
}
```

Figure 8.3 shows Rx operation flow.

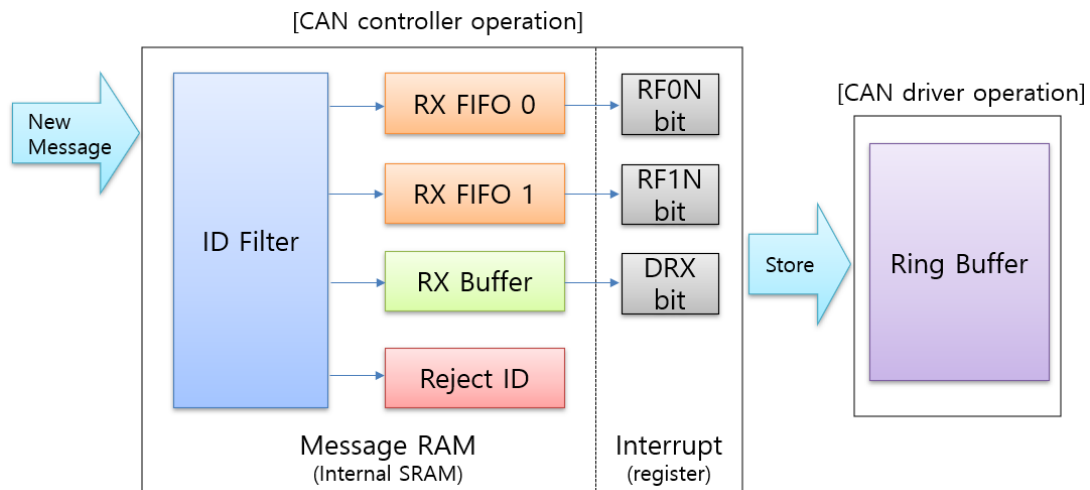


Figure 8.3 Rx Operation Flow

1. Depending on the configuration of ID filter, the received messages that are passed through the acceptance filtering are stored in one of Rx FIFO 0, Rx FIFO 1, and Rx dedicated buffer, or the received messages are rejected.
2. The CAN controller notifies the result to the CAN driver via an interrupt.
3. The CAN driver brings messages from each buffer to ring buffer by referring to the interrupt bit.
4. Application can get messages from the ring buffer.

9 REFERENCES

- [1] TCC70xx Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."